

Informe Taller 2

Docente

DELGADO SAAVEDRA CARLOS ANDRES

Estudiantes

Juan David García Arroyave

Juan José Hincapié Tascón

Sebastián Zacipa Martínez

Universidad del valle-Sede Tuluá

2024

Informe de procesos

```
class ConjuntosDifusos {  
  type ConjDifuso = Int => Double  
  
  def pertenece(elem :Int, s : ConjDifuso) : Double = {  
    s (elem)  
  }  
}
```

Esto significa que ConjDifuso se refiere a una función que toma un valor de tipo entero y lo transforma un valor de tipo double que indica el grado aceptación de ese valor en el conjunto

La función muchoMayorQue en el código se tiene en cuenta dos valores para comparar a través de un valor entre 0.0 y 1.0 determinar que diferencia hay entre dos valores

```
def muchoMayorQue(a: Int, m: Int): ConjDifuso = {  
  def mma(x: Int): Double = {  
    if (x <= a) 0.0  
    else if (x > a && x <= m) (x - a).toDouble / (m - a).toDouble  
    else 1.0  
  }  
  mma  
}
```

```
val miConjunto = muchoMayorQue(10, 20)
```

Esto significa que miConjunto es una función que toma un valor x y devuelve el grado de pertenencia de x.

Evaluar Conjunto(15)

1. Llamada a Conjunto(15), que invoca mma(15).
2. La función mma revisa el valor de x (ahora 15):

15 > 10 y 15 < 20, por lo tanto:

$$(15-10) / (20-10) = 0.5$$

Resultado: miConjunto(15) = 0.5

```
def grande(d :Int, e:Int) : ConjDifuso = {
  def grand(x :Int): Double = {
    math.pow(x.toDouble / (x+d).toDouble, e.toDouble )
  }
  grand
}
```

Evaluar

1. Conjunto grande = grande(10, 2)
2. Conjuntogrande(10)
3. La función grande evalúa $((10/(10 + 10))^2) = 0.25$

Para grande(10,2) = 0.25

La función complemento tiene como parámetro un conjunto difuso, y retorna otro conjunto difuso, que es el complemento de este.

```
def complemento( c : ConjDifuso) : ConjDifuso = {
  def complement(x :Int) : Double = {
    1.0 - c(x)
  }
  complement
}
```

Para este ejemplo se utiliza como conjunto difuso la función grande:

Conjuntogrande(5)

$\text{grande}(10, 2) = (5/(5/10))^2 = (0.333)^2 = 0.111$

Llamado a la función complemento(5)

Se realiza el cálculo, del cual obtenemos $1.0 - 0.111 = 0.889$

Esta función se encarga de recibir dos conjuntos difusos, y obtenemos un nuevo conjunto difuso que representa la unión de ambos conjuntos.

Consideremos los conjuntos $cd1 = grande(10,2)$

$Cd2 = grande(15,3)$

Llamamos la función $unión(cd1, cd2)$

$Cd1(5)$ que hace referencia a $grande(10,2)$ y para $cd2 grande(15, 3)$

Se calculo $cd1(5)$ usando $grande(10,2)$:

$$Cd1(5) = (5/(5 + 10)) ^2 = (0.333) ^2 = 0.111$$

Calculamos $cd2(5)$ usando $grande(15, 3)$

$$Cd2(5) = (5/(5 + 15)) ^3 = (0.25) ^3 = 0.015625$$

$unionConj(5)$ se calcula el valor máximo utilizando $math.max(0.111, 0.015625)$ en este caso 0.111

```
def interseccion ( cd1 : ConjDifuso , cd2 : ConjDifuso ) : ConjDifuso = {  
  def inteseccion_(i:Int): Double = {  
    math.min(cd1(i),cd2(i))  
  }  
  inteseccion_  
}
```

La función intersección toma dos conjuntos, de los cuales se retorna otro conjunto difuso que indica la intersección.

$Cd1 = grande(10,2)$

$Cd2 = grande(15, 3)$

$Intersección(cd1, cd2)$

Consideramos a $cd1$ y $cd2$

$$Cd1(5) = (5/(5 + 10)) ^2 = (0.333) ^2 = 0.111$$

$$Cd2(5) = (5/(5 + 15)) ^3 = (0.25) ^3 = 0.015625$$

En el llamado de la funcion $interseccion$, se hace uso $math.min(0.111, 0.015625)$

$interseccionConj(5)$

obtenemos como resultado 0.015625

La función inclusión tiene como objetivo revisar que un conjunto difuso está incluido en otro.

```
def inclusion( cd1 : ConjDifuso, cd2 : ConjDifuso) : Boolean = {
  def inclusion_(y:Int, max:Int) : Boolean = {
    if(y>max) true
    else if (cd1(y) > cd2(y)) false
    else inclusion_(y+1, max)
  }
  inclusion_(0,1000)
}
```

Consideramos cd1 y cd2 del ejemplo anterior

$$d1(10) = (10 / (10 + 10))^2 = 0.25$$

$$cd2(10) = (10 / (10 + 15))^3 = 0.064$$

Aquí, $cd1(10) > cd2(10)$, por lo que inclusion_ retorna false, indicando que cd1 no está incluido en cd2.

```
def igualdad(cd1 : ConjDifuso , cd2 : ConjDifuso) : Boolean = {
  inclusion(cd1, cd2) && inclusion(cd2, cd1)
}
```

Dos conjuntos difusos son iguales si cada uno está incluido en el otro (es decir, $cd1 \subseteq cd2$ y $cd2 \subseteq cd1$).

Función pertenece

La función pertenece devuelve el grado de pertenencia de un elemento elem en un conjunto difuso s. Su definición es simple: retorna el valor que resulta de evaluar $s(elem)$.

Verificación:

Como pertenece simplemente aplica la función s al elemento elem, podemos argumentar que la función cumple su propósito si s está correctamente definida para todos los elementos enteros. No requiere recursión ni iteración, por lo que la corrección es inmediata.

Función muchoMayorQue

La función muchoMayorQue(a, m) devuelve un conjunto difuso, donde cada elemento x tiene un grado de pertenencia calculado en función de su distancia a a y m. La función se define así:

Si $x \leq a$, el grado es 0.0.

Si $a < x \leq m$, el grado es una proporción que aumenta linealmente desde 0 hasta 1.

Si $x > m$, el grado es 1.0.

Verificación mediante casos:

Caso Base: Para $x \leq a$, el grado es 0.0, que es correcto según la definición.

Transición: Para $a < x \leq m$, se calcula una interpolación lineal entre 0 y 1. Esta interpolación asegura que, al incrementar x , el valor de pertenencia aumenta consistentemente de acuerdo con la distancia entre a y m .

Caso Final: Para $x > m$, el valor es 1.0, lo cual es correcto.

Por lo tanto, la función `muchoMayorQue` cumple con su especificación para todo x .

Función grande

La función `grande(d, e)` devuelve un conjunto difuso en el que el grado de pertenencia de cada elemento x depende de una potencia de su cociente respecto a d .

Verificación:

Caso Base: Cuando $x = 0$, el grado de pertenencia es 0, ya que $0/d = 0$, y al elevarlo a cualquier potencia e , sigue siendo 0.

Transición: A medida que x crece, el valor de pertenencia aumenta gradualmente. Esto cumple con el propósito de representar valores "grandes" mediante un crecimiento controlado por e , que permite ajustar la velocidad de crecimiento.

Caso Final: Para valores altos de x , el valor de pertenencia se aproxima a 1.

La función `grande` se ajusta a su especificación, calculando valores de pertenencia correctos para todos los enteros.

Función complemento

La función `complemento(c)` toma un conjunto difuso y devuelve uno donde el grado de pertenencia de cada elemento x es $1 - c(x)$.

Verificación:

Propiedad del Complemento: La operación $1 - c(x)$ asegura que cualquier valor de pertenencia en el conjunto original es invertido en su complemento. Para todos x , el resultado es un valor entre 0 y 1, preservando la propiedad de conjuntos difusos.

La corrección es inmediata dado que se cumple la propiedad de complemento.

Función union

La función $\text{union}(cd1, cd2)$ devuelve un conjunto difuso en el que el grado de pertenencia para cada elemento u es el máximo entre $cd1(u)$ y $cd2(u)$.

Verificación:

Caso Base: Si $cd1(u)$ y $cd2(u)$ son ambos 0, entonces $\text{union}(u) = 0$.

Comportamiento de Máximo: Para cada valor de u , la operación $\max$ garantiza que se tome el valor de pertenencia más alto, asegurando que cualquier elemento pertenezca al menos al nivel del conjunto donde tiene el mayor grado.

La función union cumple con su especificación, representando la unión de dos conjuntos difusos correctamente.

Función interseccion

La función $\text{interseccion}(cd1, cd2)$ devuelve un conjunto difuso en el que el grado de pertenencia para cada elemento i es el mínimo entre $cd1(i)$ y $cd2(i)$.

Verificación:

Caso Base: Si ambos conjuntos asignan un valor de 1 al mismo elemento, el resultado también será 1.

Comportamiento de Mínimo: El operador $\min$ asegura que el grado de pertenencia refleje la menor restricción entre los dos conjuntos, conforme a la definición de intersección difusa.

Así, interseccion cumple con su especificación.

Función inclusion

La función `inclusion(cd1, cd2)` verifica si cada elemento en `cd1` tiene un grado de pertenencia menor o igual al correspondiente en `cd2` en el intervalo de 0 a 1000.

Verificación:

Caso Base: Si el grado de pertenencia de cada elemento de `cd1` es igual o inferior al de `cd2` para todos los elementos hasta el límite, entonces `inclusion` retorna `true`.

Inducción: Para cada incremento en `y`, si se cumple la condición $cd1(y) \leq cd2(y)$, entonces la función recursiva mantiene la verificación hasta `y = 1000`.

La función `inclusion` verifica correctamente la inclusión de conjuntos difusos.

Función igualdad

La función `igualdad(cd1, cd2)` retorna `true` si ambos conjuntos difusos tienen exactamente los mismos grados de pertenencia en el rango definido.

Verificación:

Simetría: Comprueba la inclusión de `cd1` en `cd2` y viceversa.

Conclusión: Si ambas inclusiones son verdaderas, se cumple la igualdad entre los conjuntos difusos.

Conclusiones

Este código nos proporciona una base practica sobre conjunto difusos, teniendo a su disposición las operaciones mas comunes entre conjuntos difusos.